

How Text Becomes Numbers in NLP

Tokenization ▪ IDs ▪ Embeddings ▪ Context Windows

Your Name

2025-11-12

Why convert text to numbers?

- Neural networks operate on numbers
- Text is symbolic
- We must encode text numerically

Pipeline:

Text $\rightarrow$ Tokens $\rightarrow$ IDs $\rightarrow$ Embeddings $\rightarrow$ Tensors

Predictive Modeling Analogy

Classical ML:

$$(X_1, X_2, \dots, X_k) \rightarrow y$$

Text modeling:

$$(t_{i-2}, t_{i-1}) \rightarrow t_i$$

Next-token prediction = supervised learning on sequences.

Tokenization

Tokenization breaks text into:

- Word tokens
- Subword tokens (BPE)
- Characters

Models do **not** understand words.

Models understand **token IDs**.

Example Sentence

Sentence:

I love large language models.

Tokens:

I love large language models .

Vocabulary → Token IDs

Mapping:

Token	ID
I	1
love	2
large	3
language	4
models	5
.	6

Encoded sentence:

[1, 2, 3, 4, 5, 6]

Context Window

Assume:

- context length = 2
- predict the next token

Training examples:

$[I, \textit{love}] \rightarrow \textit{large}$

$[\textit{love}, \textit{large}] \rightarrow \textit{language}$

$[\textit{large}, \textit{language}] \rightarrow \textit{models}$

$[\textit{language}, \textit{models}] \rightarrow .$

Embedding Layer

Token IDs $\rightarrow$ dense vectors

Embedding dimension = **3**

Example embeddings:

Token	Embedding
I	(0.1, 0.2, 0.3)
love	(-0.5, 0.1, 0.0)
large	(0.7, -0.2, 0.4)
language	(0.0, 0.3, 0.9)
models	(0.4, 0.4, 0.4)
.	(-0.2, 0.0, 0.1)

Converting Tokens to Embeddings

Example input:

Tensor (shape 2×3):

$$\begin{pmatrix} 0.1 & 0.2 & 0.3 \\ -0.5 & 0.1 & 0.0 \end{pmatrix}$$

Full Training Examples

Context length = 2, embed dim = 3:

$$[1, 2] \rightarrow \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ -0.5 & 0.1 & 0.0 \end{pmatrix}$$

$$[2, 3] \rightarrow \begin{pmatrix} -0.5 & 0.1 & 0.0 \\ 0.7 & -0.2 & 0.4 \end{pmatrix}$$

$$[3, 4] \rightarrow \begin{pmatrix} 0.7 & -0.2 & 0.4 \\ 0.0 & 0.3 & 0.9 \end{pmatrix}$$

$$[4, 5] \rightarrow \begin{pmatrix} 0.0 & 0.3 & 0.9 \\ 0.4 & 0.4 & 0.4 \end{pmatrix}$$

Targets:

Next Token Prediction

The model learns:

$$f : \mathbb{R}^{2 \times 3} \rightarrow \text{Probabilities over tokens}$$

Example distribution:

- $P(\text{large}) = 0.72$
- $P(\text{language}) = 0.10$
- $P(\text{models}) = 0.08$
- $P(.) = 0.02$

Prediction = highest probability.

Regression Analogy

Text Modeling Concept	Regression Equivalent
Previous tokens	Input X
Next token	Output y
Token IDs	Encoded categorical vars
Embeddings	Learned numeric features
Context window	# predictors
Sequence model	$f(X)$

RNN/LSTM:

- Reads text sequentially
- Memory fades
- Struggles with long-range relationships

Transformers solve these.

Transformers (High-Level Summary)

Transformers add:

- Positional encoding
- Multi-head self-attention
- Feed-forward layers
- Parallelism

But the input is still the **same embedding tensor**.

Key Takeaways

- Text $\rightarrow$ tokens $\rightarrow$ IDs $\rightarrow$ embeddings $\rightarrow$ tensors
- NLP is supervised learning on sequences
- Embeddings are learned numeric features
- Context window = model's view
- LLMs predict **one token at a time**

Questions?

Thank you!